CampusClaw 项目介绍与功能验证报告

一、项目简介

CampusClaw 是面向教学场景的 AI 应用平台，基于 Flask、SQLAlchemy 和 SQLite 构建。教师和学生可以在各自班级内管理、检索资料，并使用通用对话、知识问答和解题引导三个内置应用。平台还支持提示词模板、内置及上传 Skill，以及 HTTP/MCP 工具接入。

二、功能验证

1. LLM 请求、返回内容及提示词模板

相关源码：app/platform_runtime.py

```
payload = {"model": config["model"], "messages": messages, "stream": False,
           "temperature": config["temperature"], "max_tokens": config["max_tokens"]}
if tools:
    payload.update(tools=tools, tool_choice="auto")
body = json.dumps(payload, ensure_ascii=False).encode("utf-8")

headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json", "Accept":
 "application/json"}
url = current_app.config["ZHIPU_API_URL"]

status, raw = transport(url, body, headers, timeout)

data = json.loads(raw)
choice = data["choices"][0]
message = choice["message"]

content, calls = message.get("content"), message.get("tool_calls", [])

parsed = {"content": content, "tool_calls": calls, "finish_reason":
choice.get("finish_reason"),
          "usage": data.get("usage"), "usage_provided": "usage" in data}
```

相关源码：app/platform_runtime.py

```
template = tenant(TemplateVersion, self.user, self.config["template_version_id"])
rendered = render_template_version(template, self.payload.get("variables", {}))
self.block("application_template", rendered, {"version_id": template.id})

self.block("question", self.question)
self.messages = [{"role": "system", "content": system}, {"role": "user", "content":
self.trace.redactor.clean(self.user_content)}]
```

每次调用时，后端会发送模型名称 model、消息内容 messages，以及 stream=false、temperature 和 max_tokens 等生成参数；启用工具后，还会附上 tools 和 tool_choice。提示词模板会先完成变量替换，再随消息一起发送，API Key 则放在 Authorization 请求头中。返回内容主要包括回答正文 content、工具调用 tool_calls、结束原因 finish_reason，以及接口提供的用量 usage。

2. 绑定知识库后是否发送全部内容

相关源码：app/platform_runtime.py

```
material_ids = db.session.scalars(db.select(KnowledgeBaseMaterial.material_id).where(
    KnowledgeBaseMaterial.class_id == self.user.class_id,
    KnowledgeBaseMaterial.base_id.in_(self.config["knowledge_base_ids"]))).all()
if self.config["knowledge_base_ids"]:
    self.sources = retrieve_answer_sources(self.user, self.question,
current_app.config["KNOWLEDGE_QA_MAX_CONTEXT_CHARS"], material_ids=material_ids)

for source in self.sources:
    self.block("knowledge_source", f"[{source['number']}]
{source['material_title']}\n{source['passage']}", {"chunk_id": source["chunk_id"],
"sent_end_char": source["sent_end_char"]})
```

相关源码：app/qa.py

```
selected = []
for _score, _chunk_id, chunk in ranked:
    if any(_overlaps(chunk, existing) for existing in selected):
        continue
    selected.append(chunk)
    if len(selected) == limit:

sources = []
remaining = max_context_chars
for number, chunk in enumerate(selected, start=1):
    if remaining <= 0:
        break
    passage = chunk.body[:remaining]
```

绑定知识库后，系统不会直接把全部资料交给模型，而是根据本轮问题检索相关片段，再将片段的编号、标题和正文加入请求。默认最多选取 4 段，正文总长度不超过 6000 字符。

3. 绑定 Skill 后是否发送全部内容

相关源码：app/platform_runtime.py

```
catalog = []
for ident, version in self.skill_versions.items():
    package = tenant(SkillPackage, self.user, version.package_id, True)
    catalog.append({"id": ident, "name": package.name, "description": version.description,
"version": version.version, "digest": version.digest})
if catalog:
    self.block("skill_catalog", json.dumps(catalog, ensure_ascii=False))
selected = self.payload.get("skill_version_id")
if selected is not None:
    self.trace.event("skill_selected", {"skill_version_id": selected})
    body = self.load_skill(selected)
    self.block("active_skill", body, {"version_id": selected})

return version.body

return reference.content
```

绑定 Skill 并不意味着每次都发送整个技能包。系统会先提供名称、描述等目录信息；用户选中技能或模型调用 load_skill 时，再加入主指令正文。参考文件则通过 read_skill_reference 按需读取，不会一次性全部传入。

4. 执行具体 Skill 的逻辑

相关源码：app/platform_runtime.py

```
if name == "load_skill":
    from app.platform_tools import validate_arguments
    validate_arguments(builtin_tools()[0]["function"]["parameters"], arguments)
    return self.load_skill(arguments["skill_version_id"])

for version in self.loaded.values():
    if version.allowed_tools is not None and name not in version.allowed_tools and
definition.name not in version.allowed_tools:
        raise PlatformError("skill_tool_forbidden", 403)

message, call_id = provider_call(trace, self.config, self.messages, self.tools, self.api_key,
self.remaining())

output = self.tool(name, arguments)
```

技能选定后，后端会加载主指令，由模型据此回答，或提出读取参考文件、调用工具的请求。后端完成权限和参数校验后执行相应操作，再把结果交回模型，继续生成答案。这里的 Skill 以指令为主，不直接运行 Python 或 Shell 脚本。

5. MCP 集成后发送给 LLM 的参数

相关源码：app/platform_runtime.py

```
name = f"ext_{connection.id}_{definition.id}"
self.external[name] = definition

self.tools.append({"type": "function", "function": {"name": name, "description":
self.trace.redactor.clean(definition.description), "parameters":
self.trace.redactor.clean(definition.input_schema)}})

if tools:
    payload.update(tools=tools, tool_choice="auto")

self.messages.append(trace.redactor.clean(copy.deepcopy(message)))

content = clean if isinstance(clean, str) else json.dumps(clean, ensure_ascii=False)
self.messages.append({"role": "tool", "tool_call_id": call["id"], "content": content})
```

相关源码：app/platform_tools.py

```
result = mcp_operation(connection, "tools/call", {"name": definition.name, "arguments":
arguments}, remaining)
```

接入 MCP 后，请求会在基础参数之外增加 tools 和 tool_choice="auto"，向模型提供工具的类型、名称、用途和参数格式。模型通过 tool_calls 提出调用请求后，由后端执行 MCP 工具，再将调用记录和结果加入下一次请求；结果消息使用 role="tool"，并通过 tool_call_id 与对应调用关联。MCP 服务地址和认证配置由后端使用，不会自动传给模型。